S-VAULT

Personal Vault Security System (PVSS)

A CLI-Based Password Manager Built with Python

Hassan | February-March 2026
GitHub: https://github.com/h14san/S-Vault

Abstract

S-Vault is a command-line interface (CLI) password manager developed in Python, designed to address the growing need for lightweight, offline, and secure personal credential management. The system authenticates users through a master password protected by bcrypt hashing and derives an encryption key using PBKDF2HMAC with SHA-256. All stored credentials are encrypted using the Fernet symmetric encryption scheme from Python's cryptography library, and persisted locally in JSON format. S-Vault additionally features a cryptographically secure password generator based on Python's secrets module. This paper describes the motivation, design, security architecture, implementation details, and potential future improvements of the system.

---

1. Introduction

Password security remains one of the most critical yet neglected aspects of personal cybersecurity. The average user manages dozens of accounts across various platforms, often resorting to weak, reused passwords or relying on cloud-based managers that introduce additional trust assumptions. While commercial tools such as LastPass or Bitwarden offer convenience, they require internet connectivity, account creation, and trust in a third-party server.

S-Vault was built to offer a different philosophy: a fully offline, terminal-based vault that keeps all credentials on the user's own machine and requires no external services. The project prioritizes simplicity, transparency, and solid cryptographic foundations while remaining accessible to technical users comfortable with command-line tools.

The key goals of S-Vault are:

- Provide strong encryption for stored passwords using industry-standard algorithms.
- Authenticate users securely through bcrypt-hashed master password verification.
- Enable cryptographically secure password generation.
- Remain fully offline and local, no network calls, no cloud dependency.

- Keep the codebase minimal, readable, and auditable.

---

2. Background & Motivation

2.1 The Password Management Problem

According to security researchers, password reuse and weak passwords remain the leading causes of account compromise. Users are advised to use unique, complex passwords for every service, a task humanly impossible without tooling. Password managers solve this by storing credentials securely and retrieving them on demand.

2.2 Limitations of Existing Tools

Cloud-based managers (e.g., LastPass, 1Password) require network access and trust in the provider's infrastructure. Several high-profile breaches have demonstrated that centralized vaults are attractive targets. Local alternatives often sacrifice usability or require GUI environments. S-Vault targets developers and security-conscious users who prefer a transparent, CLI-native solution they can inspect and control entirely.

---

3. System Architecture & Design

3.1 Overview

S-Vault follows a two-module architecture split into svault.py (the main entry point and CLI loop) and resources.py (all cryptographic and storage logic). This separation ensures that the application layer remains decoupled from security-sensitive operations.

3.2 File Structure

File	Purpose
svault.py	Main CLI loop; user interaction and menu navigation
resources.py	All cryptographic operations, password storage, and utilities
passwords.json	Encrypted password store (Fernet ciphertext, app-keyed)
master.hash	bcrypt hash of the master password
requirements.txt	Python dependency declarations

3.3 User Flow

The application follows a straightforward authentication-first flow:

- On launch, the user is prompted to login or create an account.
- Account creation: the master password is bcrypt-hashed and written to master.hash, and an empty passwords.json is initialized.
- Login: the entered password is verified against the stored hash. On success, an encryption key is derived from the password using PBKDF2HMAC.
- Post-login, the user can add a password manually, generate a strong password, view all saved app names (masked), or search and decrypt a specific password.
- The derived key never leaves memory and is not stored to disk.

---

4. Security Architecture

4.1 Master Password Hashing — bcrypt

When a user creates an account, the master password is hashed using bcrypt with a randomly generated salt. bcrypt is an adaptive hashing function specifically designed for password storage: it is computationally expensive by design, making brute-force attacks significantly slower. The resulting hash is stored in master.hash and compared against the entered password at each login using bcrypt.checkpw().

4.2 Key Derivation — PBKDF2HMAC with SHA-256

Upon successful login, S-Vault derives a 32-byte symmetric encryption key from the master password using PBKDF2HMAC (Password-Based Key Derivation Function 2) with SHA-256 as the underlying hash and 100,000 iterations. This key derivation function ensures that the encryption key is deterministically derived from the password, meaning the vault can be decrypted without storing the key anywhere. The derived key is encoded in Base64 URL-safe format for use with Fernet.

Note: the current implementation uses a static salt (b'svault'). A future improvement would store a unique random salt per installation to further harden against precomputed attacks.

4.3 Password Encryption — Fernet (AES-128-CBC + HMAC-SHA256)

All stored passwords are encrypted using Fernet, a symmetric encryption scheme from Python's cryptography library. Under the hood, Fernet uses AES-128 in CBC mode for encryption and HMAC-SHA256 for authentication, ensuring both confidentiality and integrity of stored data. Each encrypted value is stored as a string in passwords.json under the corresponding app name. Decryption requires the Fernet key derived from the master password, which means the vault file is useless without the correct master password.

4.4 Password Generation — Python secrets Module

S-Vault's password generator uses Python's secrets module, which provides cryptographically secure random numbers suitable for generating tokens and passwords. The generator ensures that every generated password contains at least one uppercase letter, one lowercase letter, one digit, and one special character. The remaining characters are drawn randomly from the full character set, and the final password is shuffled using secrets.SystemRandom() to eliminate any positional bias.

4.5 Secure Input

The master password is always collected using Python's getpass module, which suppresses terminal echo and prevents the password from appearing on screen during entry.

---

5. Implementation Details

5.1 Technologies & Dependencies

Library / Tool	Role
Python 3	Core language
bcrypt	Master password hashing
cryptography (Fernet, PBKDF2HMAC)	Key derivation and symmetric encryption
secrets	Cryptographically secure password generation
getpass	Silent password input
json	Persistent local storage
string	Character set for password generation

5.2 Core Functions (resources.py)

- create_account(password): Hashes master password with bcrypt and initializes the vault.
- login(password): Verifies the entered password against the stored bcrypt hash.
- derive_key(master_password): Runs PBKDF2HMAC to produce the Fernet encryption key.
- save_password(app, password, key): Encrypts a credential with Fernet and persists it.
- load_password(): Reads the encrypted vault from passwords.json.
- search_password(app, master_key): Decrypts and returns a specific credential.
- generate_password(n): Generates a secure n-character password with guaranteed character diversity.

---

6. Limitations & Future Improvements

6.1 Current Limitations

- Static PBKDF2 salt: The salt b'svault' is hardcoded. A random per-installation salt stored alongside the hash would improve resistance to precomputed dictionary attacks.
- Single user: The vault is designed for a single master password and credential store. Multi-user or multi-profile support is not implemented.
- No clipboard integration: Passwords must be read from the terminal. Clipboard copy would improve usability.
- No password update or deletion: Users cannot currently edit or remove a saved credential.
- No session timeout: The vault remains unlocked for the entire session with no idle timeout.

6.2 Planned Improvements

A companion improvements.txt file in the repository documents the following planned enhancements:

- Random per-installation salt for PBKDF2HMAC.
- Add/update/delete operations for stored credentials.
- Optional clipboard integration (e.g., via pyperclip).
- Session timeout after a configurable idle period.
- Password strength checker for manually entered credentials.
- Export/import functionality for backup purposes.

---

7. Conclusion

S-Vault demonstrates that a secure, functional password manager can be built with a minimal Python codebase by relying on well-established cryptographic libraries. The system combines bcrypt for authentication, PBKDF2HMAC for key derivation, and Fernet for encryption; all industry-standard approaches, without any external server dependency.

The project serves as both a practical tool and a learning exercise in applied cryptography. Its open-source nature means the security model is fully auditable, and future iterations can address the identified limitations to bring it closer to production-readiness.

S-Vault is available on GitHub at https://github.com/h14san/S-Vault.

References

[1] Python Software Foundation. Secrets; Generate secure random numbers for managing secrets. https://docs.python.org/3/library/secrets.html

[2] Python Cryptographic Authority. Cryptography; Fernet (symmetric encryption). https://cryptography.io/en/latest/fernet/

[3] Python Cryptographic Authority. PBKDF2HMAC. https://cryptography.io/en/latest/hazmat/primitives/key-derivation-functions/#cryptography.hazmat.primitives.kdf.pbkdf2.PBKDF2HMAC

[4] Niels Provos and David Mazières. A Future-Adaptable Password Scheme. USENIX Annual Technical Conference, 1999.

[5] NIST Special Publication 800-132. Recommendation for Password-Based Key Derivation. National Institute of Standards and Technology, 2010.